

Platform as a Service. Casos de uso en Vercel y App Engine

Cloud Computing - Máster en Ingeniería Informática

Paco García y Manuel Torres <Paco.Garcia@ual.es> <mtorres@ual.es>



DEPARTAMENTO
DE INFORMÁTICA
UNIVERSIDAD DE ALMERÍA

Resumen

El modelo PaaS (Platform as a Service) de cloud computing es un modelo muy cómodo para el desarrollo. En PaaS el proveedor cloud ofrece un entorno o plataforma en el que el equipo de desarrollo puede crear y distribuir aplicaciones sin preocuparse de la infraestructura. El proveedor se encarga del mantenimiento y actualización de la plataforma. Además, ofrece servicios complementarios como bases de datos, monitorización, logging permitiendo centrarse en el desarrollo y despreocupándose de la actualización de versiones del entorno de ejecución, librerías, parches de seguridad y demás.

En este tutorial se realiza una introducción al modo PaaS y se ilustra su funcionamiento desplegando una aplicación sencilla en Vercel y Google App Engine.

Objetivos

- Conocer las ventajas que propociona PaaS.
- Distinguir los modelos IaaS y PaaS.
- Aprender a desplegar aplicaciones en Vercel.
- Utilizar las opciones de despliegue automático y *roll back* de Vercel.
- Aprender a desplegar aplicaciones en Google App Engine.

Chapter 1. Introducción

Dentro de los modelos de servicio de cloud computing, en PaaS (Platform as a Service) el proveedor ofrece un entorno o plataforma en el que los usuarios pueden crear y distribuir aplicaciones sin preocuparse de la infraestructura. Es el proveedor el que proporciona el Sistema operativo, software, librerías, gestión de certificados SSL y demás, todo en su sin preocuparnos de las actualizaciones. Es el proveedor el que se encarga de su mantenimiento. De esta forma, **el modo PaaS permite a los desarrolladores centrarse en el desarrollo. Es ideal para realizar pruebas y crear prototipos de nuevas aplicaciones**

Además los proveedores PaaS suelen ofrecer gran cantidad de servicios que pueden ser consumidos desde las aplicaciones (bases de datos, herramientas de monitorización y logging, almacenamiento, despliegue continuo, y un largo etcétera).

NOTE

En el modo IaaS (Infrastructure as a Service) el proveedor proporciona almacenamiento, redes, máquinas virtuales y otros servicios virtuales. Es el usuario el encargado del mantenimiento del sistema operativo y todo el software, paquetes y librerías que se instalen sobre él

Chapter 2. Vercel

[Vercel](#) es un PaaS que permite el despliegue de aplicaciones estáticas (HTML, CSS, JavaScript) y aplicaciones de servidor (Node.js, Go, PHP, Python, Ruby, Java, Docker).

Actualmente Vercel ofrece plataforma para desarrollo en:

- Node.js
- Edge
- Go
- Python
- Ruby
- PHP
- Bash
- Deno
- Rust

2.1. Registro en el plan gratuito de Vercel

Vercel ofrece varios planes comerciales y uno gratuito. El gratuito se llama **Hobby** y es útil para pruebas de concepto y entre otras cosas ofrece despliegue desde GitHub con despliegue continuo y bases de datos PostgreSQL. Por tanto, es una opción muy atractiva para usarlo aquí.

Regístrate con una cuenta de email seleccionando **Sign up** en la esquina superior derecha. Lo primero que debes seleccionar es el tipo de plan. Selecciona **Hobby**, introduce tu nombre y pulsa **Continue**.



Contact [Log In](#)

Create Your Vercel Account

Plan Type

Hobby
I'm working on personal projects

Pro
I'm working on commercial projects

Your Name

Continue

A continuación, puedes utilizar una cuenta de GitHub o GitLab para registrarte o una dirección de email. Selecciona la opción que prefieras y completa el registro.

Por último, se te pedirá que confirmes tu número de teléfono. Introduce tu número de teléfono y pulsa **Continue**. Recibirás un SMS con un código que deberás introducir en el campo **Code** y pulsar **Verify**.

Verification

Please enter your phone number to verify your login.
You will receive a single use code to submit.

 ▾ +34

We won't ever message you — other than for verification purposes.

Continue

Chapter 3. Despliegue en Vercel

A continuación vamos a ver cómo desplegar una aplicación PHP desde un repositorio GitHub activando la opción de despliegue continuo. El despliegue continuo permitirá la actualización de la aplicación de forma desastitida tras una subida de cambios (a una rama) a un repositorio.

3.1. Creación de un fork de la aplicación

Desde tu cuenta de usuario GitHub, crea un fork del [repositorio GitHub de la aplicación de ejemplo](#).

Se trata de una aplicación muy sencilla, con el único fin de servir como prueba de concepto de como el modelo PaaS facilita centrarse en el desarrollo, haciendo transparente al equipo de desarrollo la gestión de la plataforma subyacente (en este caso intérprete PHP, servidor web, librerías, almacenamiento y otros detalles de infraestructura).

+ El descriptor `vercel.json`

La configuración de las aplicaciones de Vercel se realiza mediante un archivo denominado `vercel.json`. En este archivo se especifican, entre otros, el entorno de ejecución, los controladores para atender las peticiones de la aplicación y los archivos estáticos (p.e. imágenes).

A continuación se muestra un ejemplo de archivo `vercel.json`.

```
{
  "functions": {
    "api/index.php": { ①
      "runtime": "vercel-php@0.6.0" ②
    }
  },
  "routes": [
    { "src": "/*", "dest": "/api/index.php" } ③
  ]
}
```

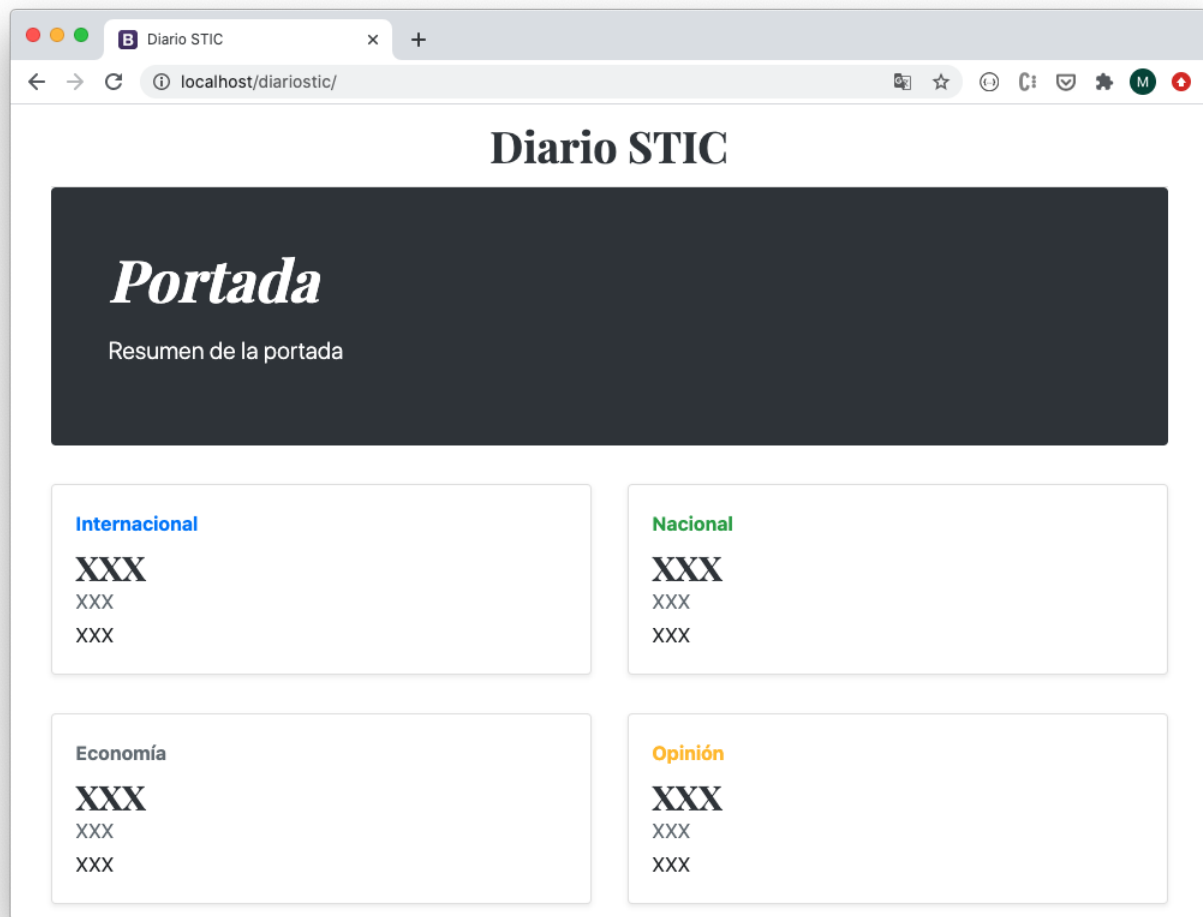
① Controlador principal para servir la aplicación

② Entorno de ejecución

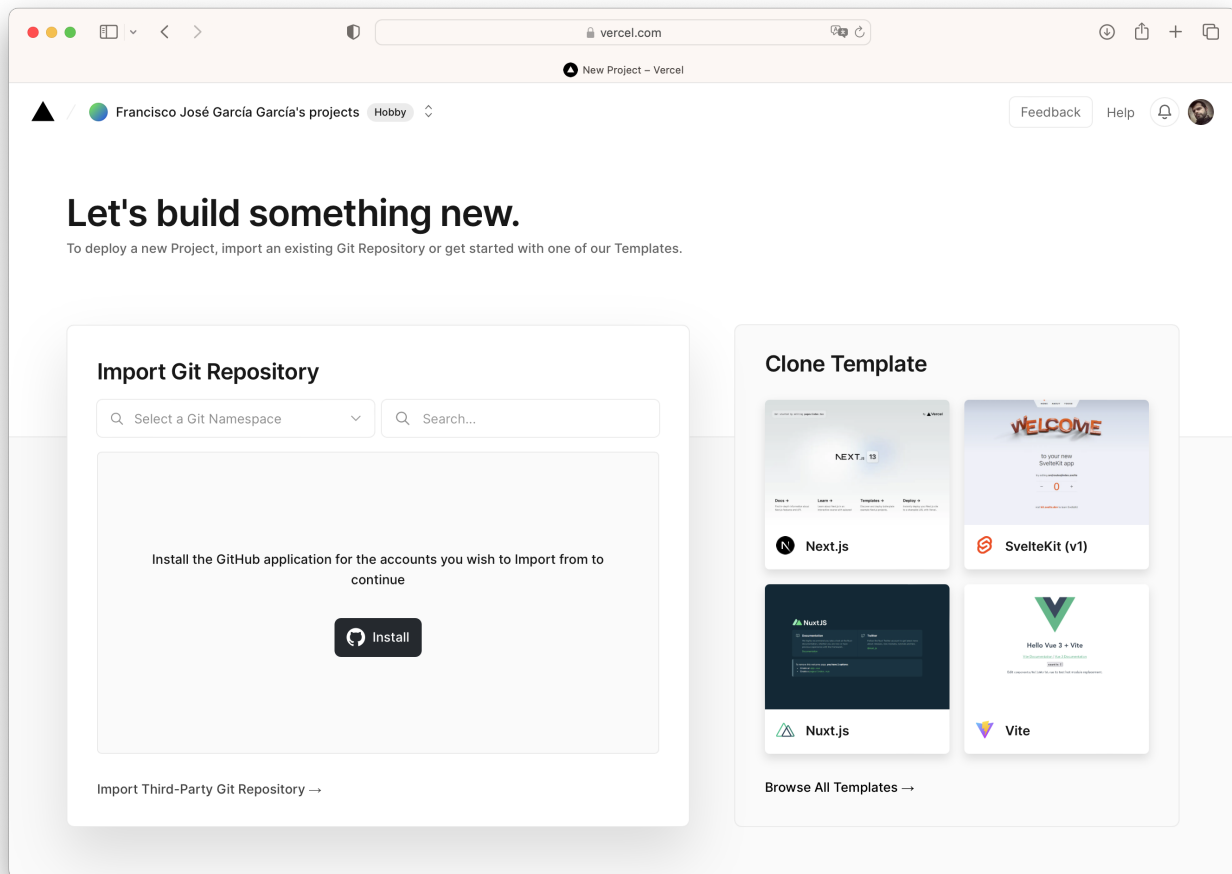
③ Asociación de rutas a controladores NOTE]

Hay que indicar cómo se gestionan cada una de las rutas de la aplicación en cuanto a código. Esto se realiza mediante elementos en el elemento `functions`. Estos deben de estar en la carpeta `api` de la aplicación.

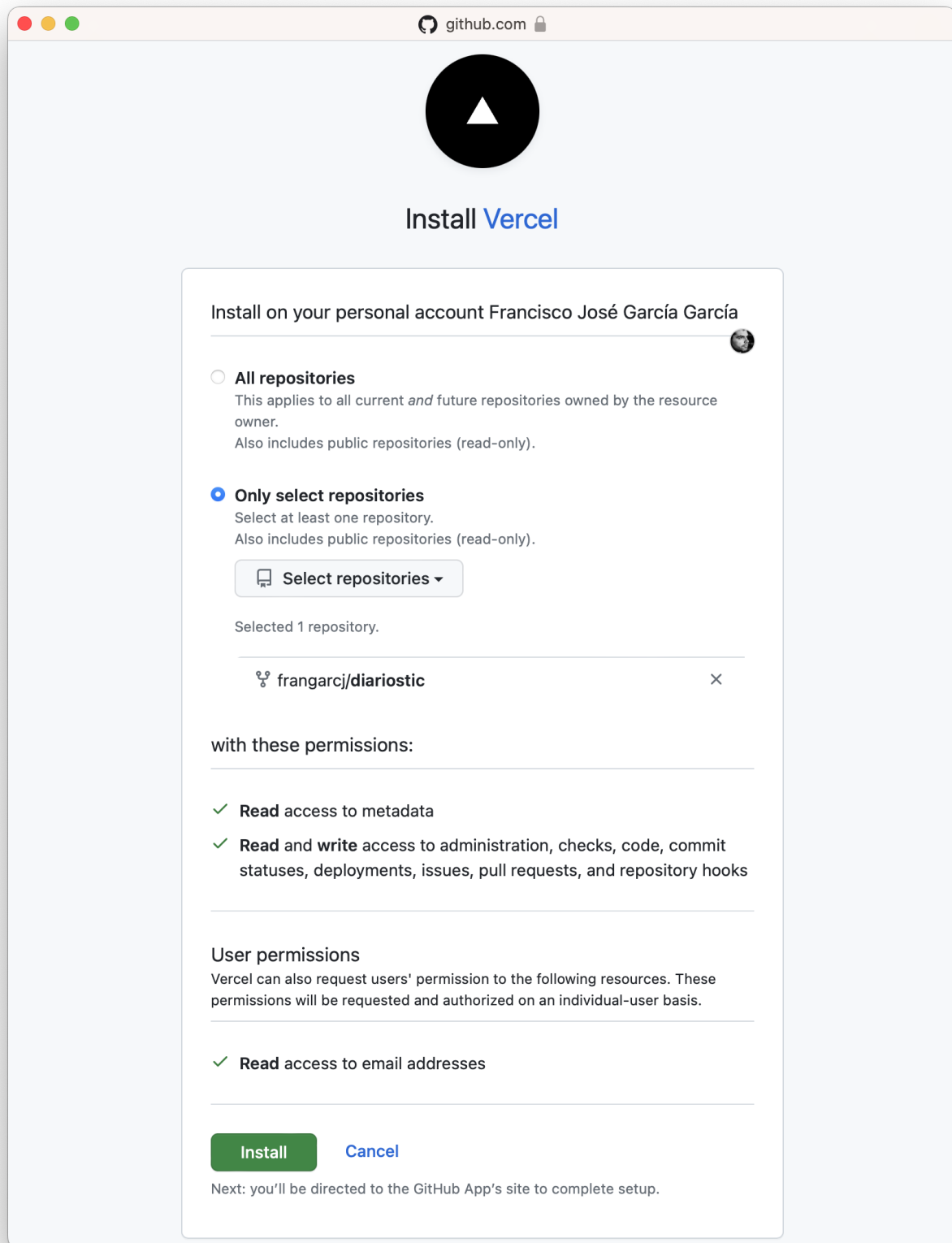
Más información en la [documentación oficial](#).



En la pantalla inicial (projects) de Vercel seleccionar **Import Git Repository | Continue with GitHub | Install.**



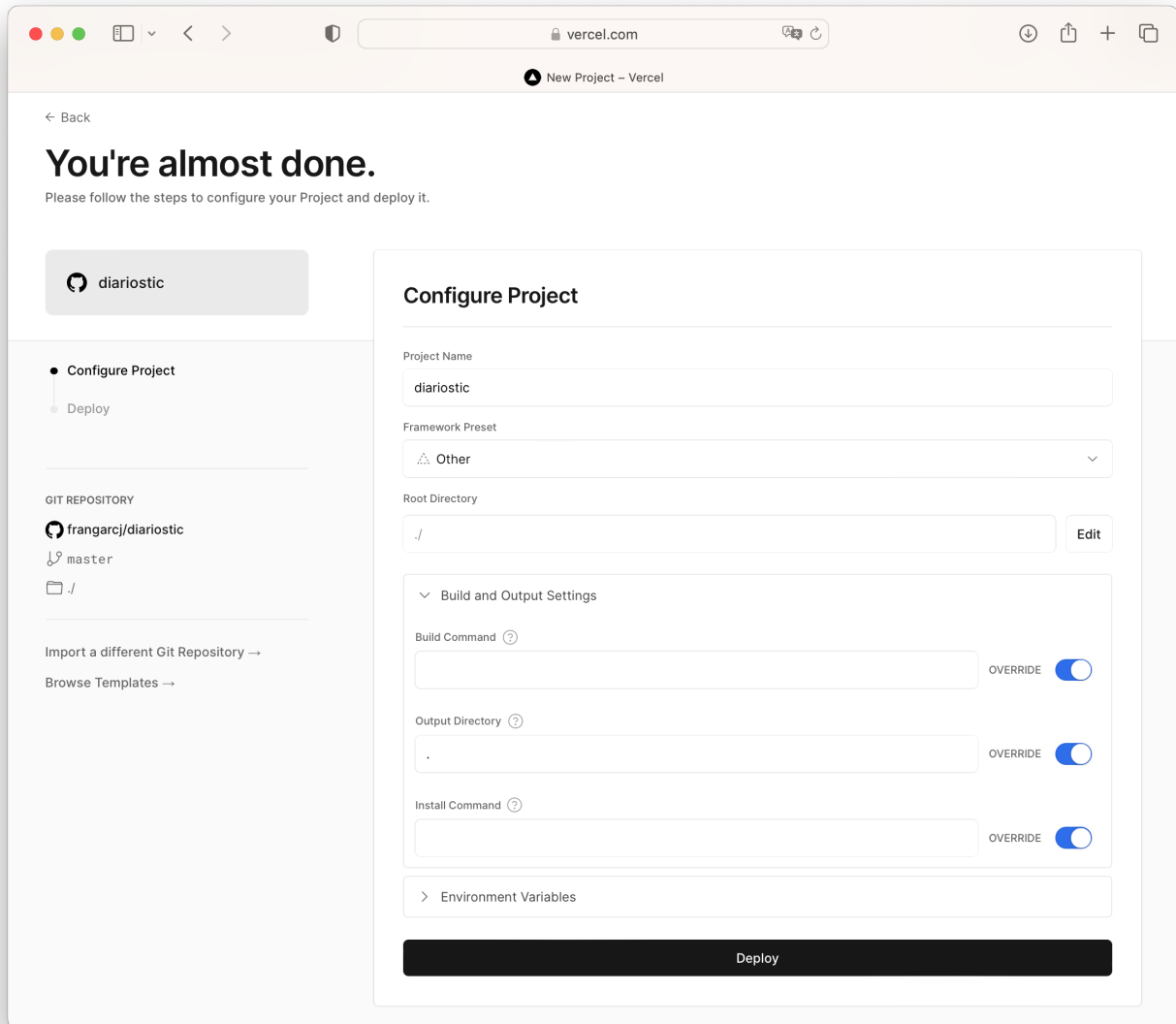
Instala Vercel en tu cuenta de GitHub y selecciona el repositorio **diariotic** que acabas de hacer fork.



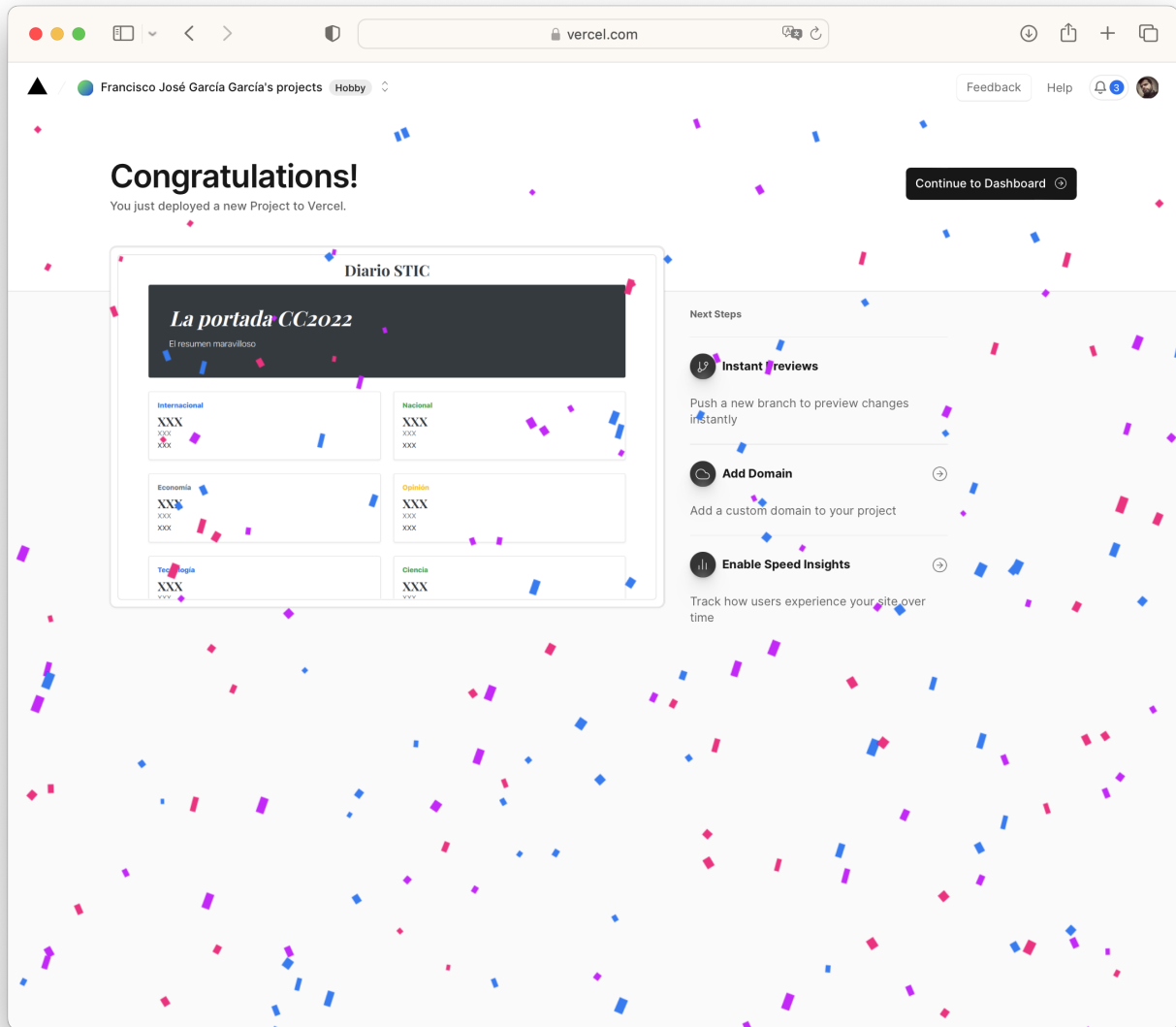
A continuación, selecciona el repositorio **diariostic** y pulsa **Import**. La siguiente pantalla muestra información sobre la configuración del proyecto. En este caso solo hay que establecer un nombre para la aplicación y pulsar **Deploy**.

NOTE

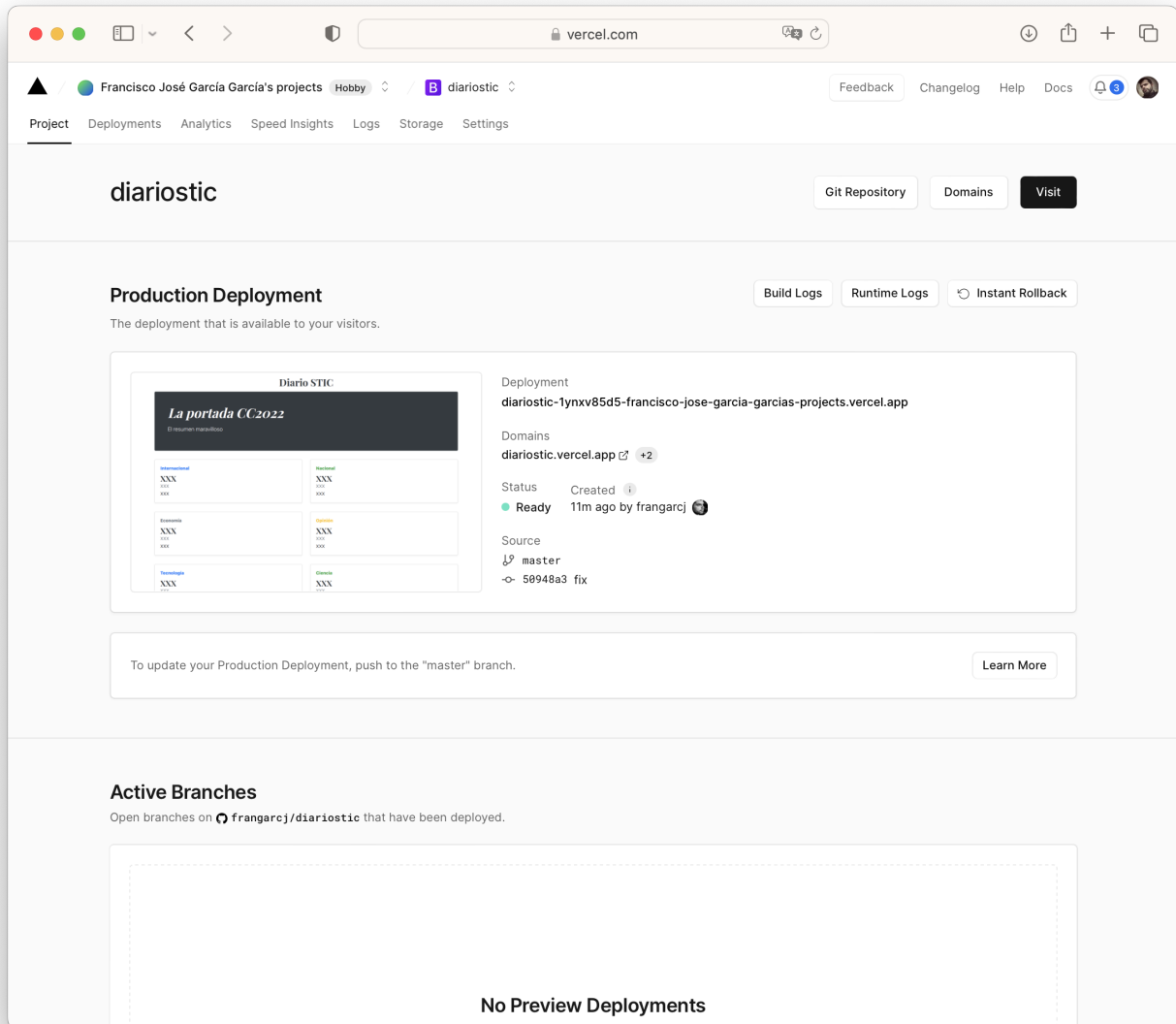
El nombre elegido para la aplicación ha de ser único ya que es el que usará para componer el nombre DNS de la aplicación.



Tras realizar el despliegue, se mostrará una preview de la aplicación. Pulsar sobre ella para ver la aplicación desplegada.



La opción de despliegue automático está activa por defecto y permite actualizar la aplicación tras efectuar **push** en la rama indicada del repositorio. En este caso se trata de la rama **master** del repositorio.



3.2. Despliegue automático

Realizar un **push** sobre el repositorio editando directamente el archivo `src/secciones/portada.php` cambiando el título por **Portada actualizada**.

```
<?php

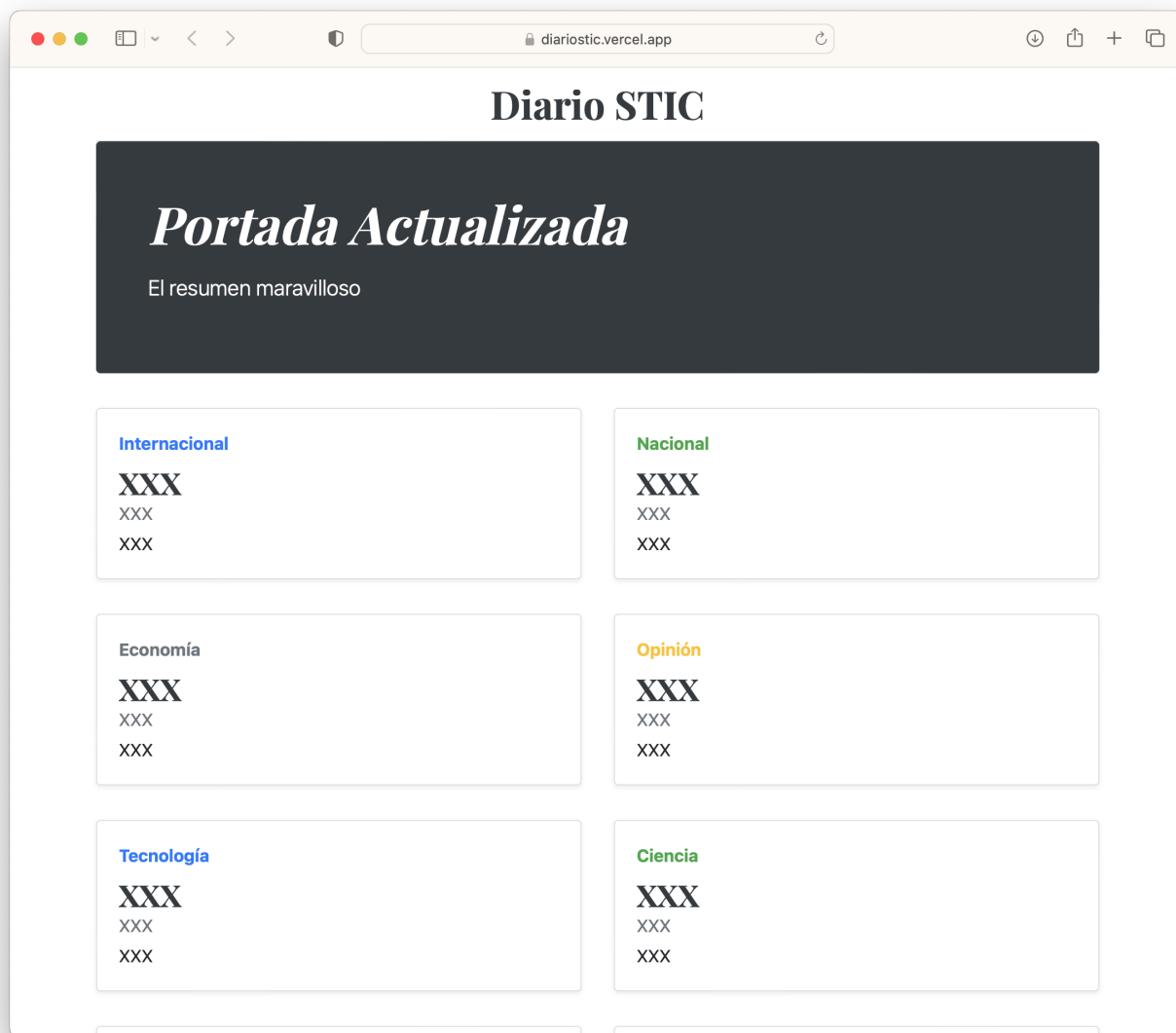
$portada = [
  "titulo" => "Portada actualizada", ①
  "autor" => "mtorres",
  "resumen" => "Resumen de la portada",
];
?>
```

① Cambiar el título

Añade un mensaje al commit (p.e. **Cambio del título de la portada**) y realiza el commit.

Tras unos instantes la aplicación quedará actualizada de forma automática con el nuevo título de la

portada.

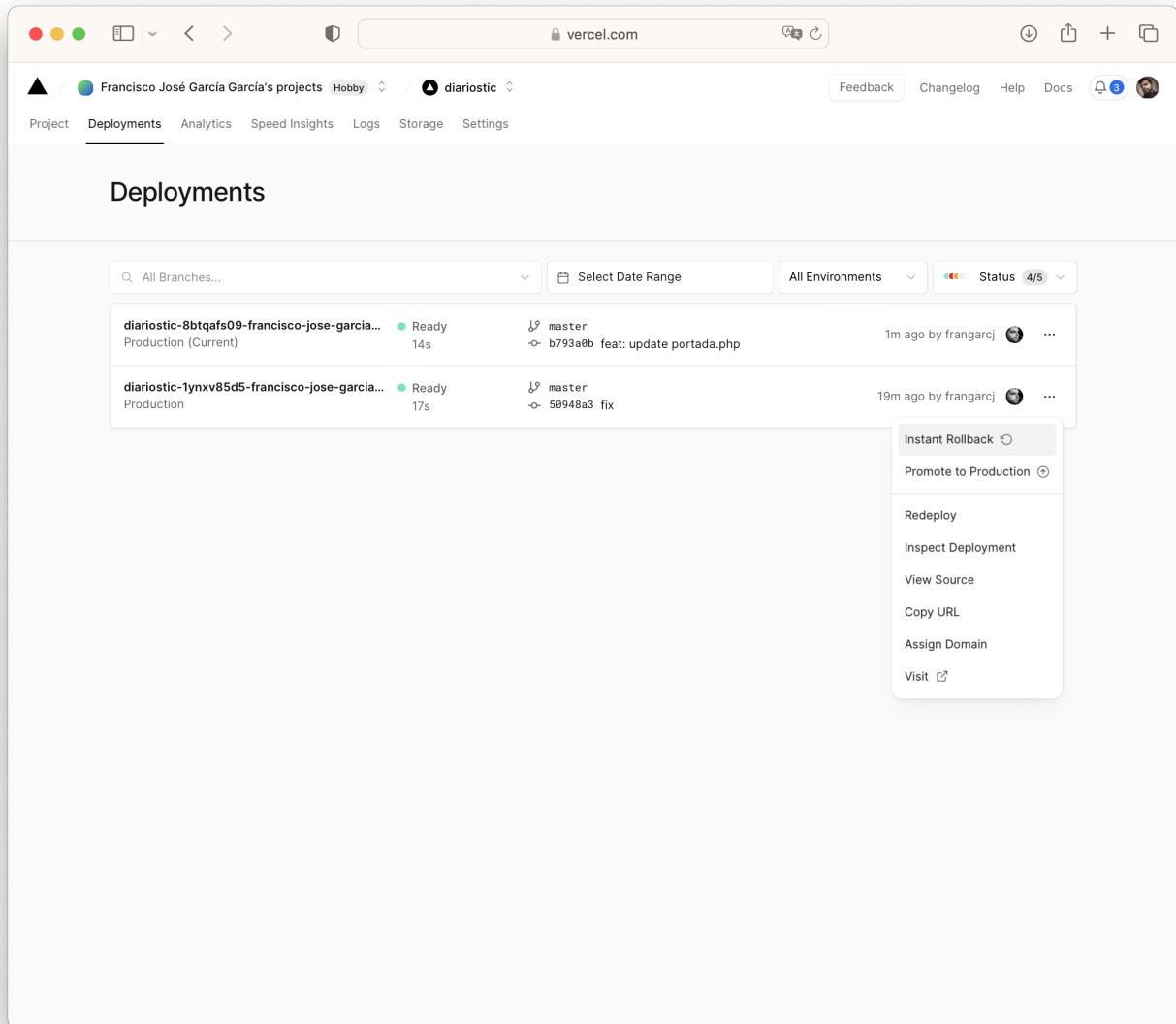


3.3. Roll back

Es posible volver a estados anteriores de la aplicación. Por ejemplo, tras desplegar un cambio comprobamos que la aplicación no funciona correctamente y se necesita volver atrás mientras se resuelve el problema.

La pestaña **Deployments** de la aplicación contiene una lista de toda la actividad llevada a cabo en la aplicación (despliegue de commits, operaciones de build, ...).

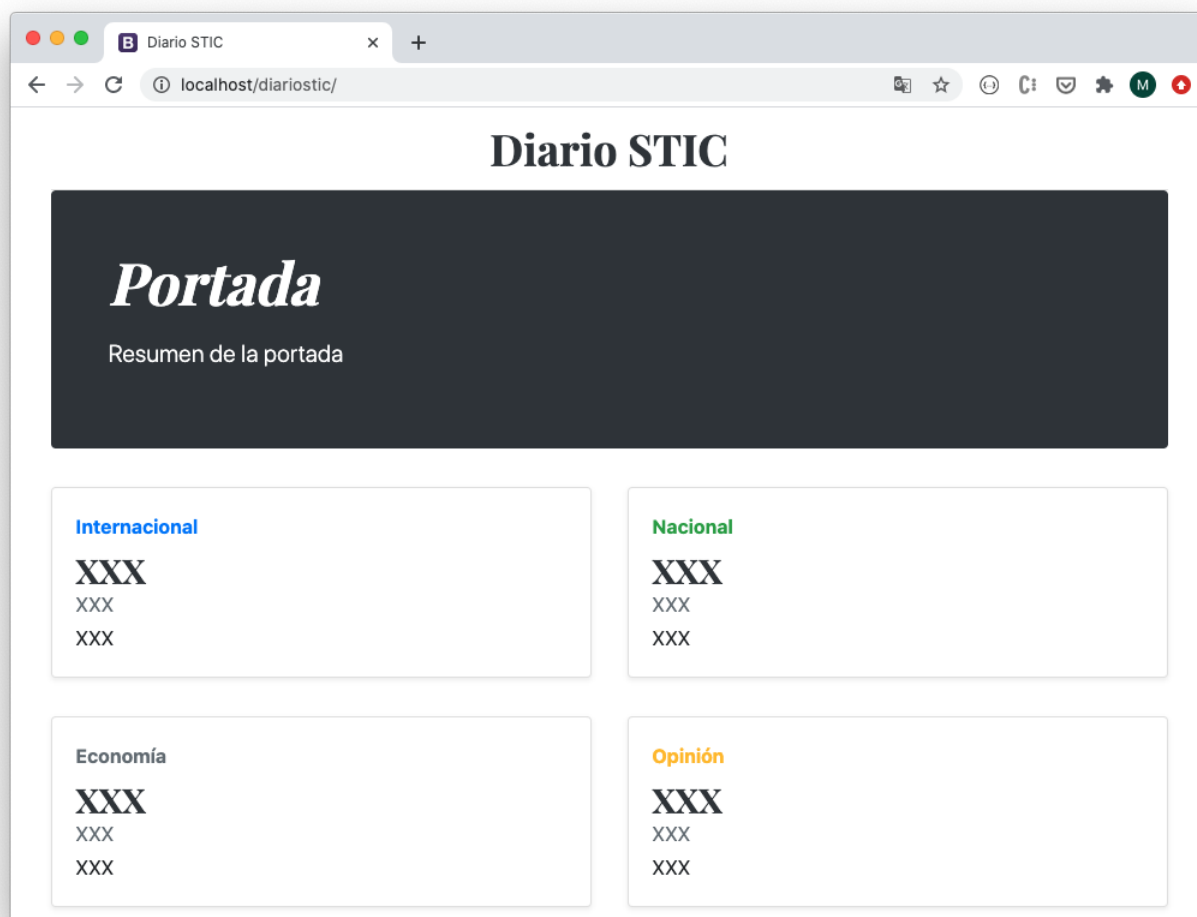
Los despliegues se pueden reconocer porque tienen asociado el hash del commit que tomaron como base. En las opciones de los puntos suspensivos podemos encontrar la opción de **Instant Rollback**. Pulsar sobre el que aparece en el primer commit.



Aparecerá un cuadro de mensaje pidiendo la confirmación para volver a la versión anterior. Pulsar **Continue**.

[VercellRollbackConfirmacion] | *images/VercellRollbackConfirmacion.png*

Se creará una nueva versión, que aparecerá en el registro de actividad y la aplicación volverá a su estado original tras unos instantes.



3.4. Otras opciones de interés

En las pestañas de la aplicación se pueden encontrar otras opciones de interés como Analytics, Speed Insights, Logs y sobre todo Storage.

En esta última se pueden crear diferentes bases de datos como KV, Postgres o almacenamiento de ficheros Blob.

NOTE

Hay que tener en cuenta que el plan gratuito **Hobby** tiene limitaciones en cuanto a número de bases de datos y almacenamiento. Consultar la [documentación oficial](#) para más información.

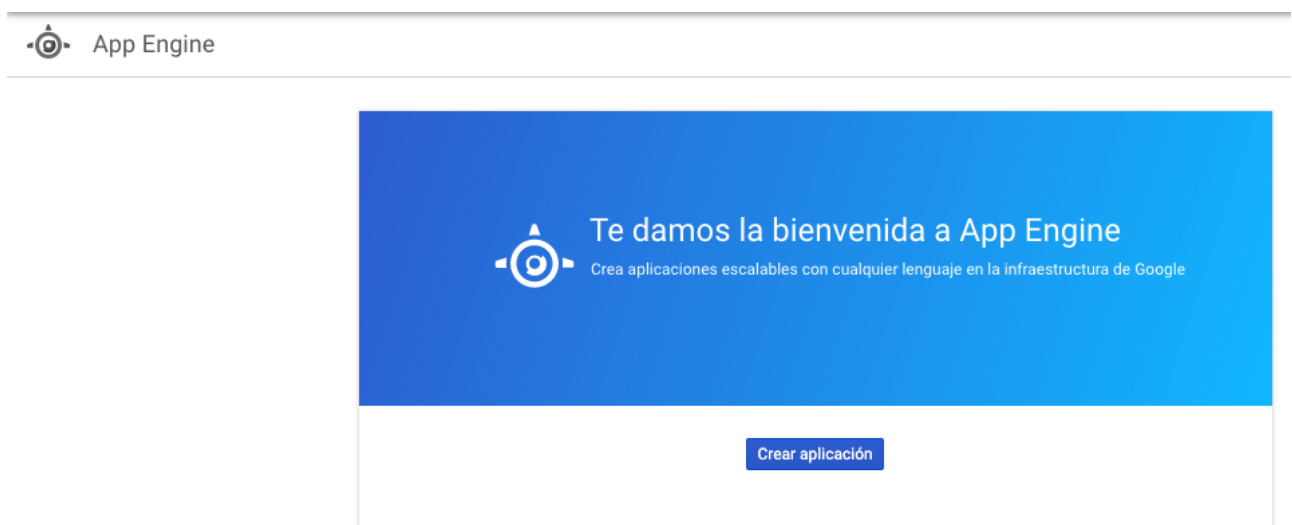
Chapter 4. Despliegue en Google App Engine

Google App Engine es el PaaS de Google. Permite el desarrollo de aplicaciones en una plataforma totalmente gestionada por Google. Actualmente App Engine ofrece plataforma para desarrollo en:

- Node.js
- Ruby
- Java
- C#
- PHP
- Python
- Go

Google App Engine permite el escalado de aplicaciones y distribución por todas las regiones sin tener que preocuparnos de la infraestructura. Se trata de un despliegue sin configuración de servidor lo que supone gran agilidad.

1. En el Menú de navegación, seleccionar **App Engine**. Aparecerá una pantalla de bienvenida. Pulsar **Crear Aplicación**.



2. Indicar la región de despliegue. Elegiremos **europa-west** y pulsaremos sobre **Crear aplicación**.
3. Seleccionar el lenguaje en el que está creada la aplicación para que Google App Engine prepare la plataforma de ejecución de la aplicación. En este caso elegiremos PHP.



i Este paso es opcional y su función es guiarte al SDK y a los ejemplos de código pertinentes y, si es necesario, habilitar la facturación.

Language

PHP

Environment

Cada versión de tu aplicación puede usar el tiempo de ejecución estándar o flexible. Puedes cambiarlo más adelante.

Estándar

Siguiente

Cancelar

Tras unos instantes se informa que la aplicación se ha creado con éxito.

**Recursos**[Documentation](#) for PHP in App Engine.Visit [Github](#) for code samples in standard PHP**Desplegar con el SDK de Google Cloud****Descargar el SDK de Google Cloud**

Inicializar el SDK

```
$ gcloud init
```

Desplégala en App Engine

```
$ gcloud app deploy
```

[LO HARÉ MÁS ADELANTE](#)

Tu aplicación de App Engine se ha creado correctamente



Se nos ofrecen recursos para la documentación de App Engine en el lenguaje usado y ejemplos de código en GitHub. A la derecha tenemos los comandos que hay que ejecutar en Cloud Shell y que ejecutaremos en unos instantes. Por ahora, saldremos del asistente pulsando **LO HARÉ MAS ADELANTE**.

4. Abrir Cloud Shell
5. Clonar el repositorio de la aplicación

```
$ git clone https://github.com/ualmartorres/diariostic.git
```

6. Crear el descriptor **app.yaml** en la carpeta del código de la aplicación

```
$ cd diariostic
$ wget
https://gist.githubusercontent.com/ualmartorres/5f4c4f4502c9fc9f84c967a59ec4203e/raw/
```

El descriptor `app.yaml`

La configuración de las aplicaciones de App Engine se realiza mediante un archivo denominado `app.yaml`. En este archivo se especifican, entre otros, el entorno de ejecución, los controladores para atender las peticiones de la aplicación y los archivos estáticos (p.e. imágenes).

A continuación se muestra un ejemplo de archivo `app.yaml`.

```
runtime: php55 ❶
api_version: 1
handlers:
- url: / ❷
  script: api/index.php
# Serve images as static resources.
- url: /\.(gif|png|jpg))$ ❸
  static_files: \1
  upload: .+\.(gif|png|jpg)$
  application_readable: true
# Serve php scripts.
- url: /\.(.php)$
  script: \1
```

❶ Entorno de ejecución

❷ Controlador principal para servir la aplicación

❸ Controlador para servir archivos estáticos

NOTE

Hay que indicar cómo se gestionan cada una de las rutas de la aplicación en cuanto a código, imágenes, CSS, JavaScript. Esto se realiza mediante elementos `url` en el elemento `handlers`.

Más información en la [documentación oficial](#).

7. Ver la aplicación en modo de prueba

```
$ php -S localhost:8080 -t api
```

8. Abrir `Vista previa web`

9. Inicializar en Cloud Shell el SDK de Google Cloud

```
$ gcloud init
```

Para poder usar el SDK de Google Cloud en Cloud Shell, Google Cloud nos advierte que necesita

incluir nuestras credenciales en Cloud Shell. Pulsaremos **Autorizar**.

Autorizar a Cloud Shell

gcloud requiere tus credenciales para hacer una llamada a la API de GCP.

Haz clic para autorizar esta llamada y las que se realicen en el futuro que requieran tus credenciales.

AutorizarRechazar

10. Desplegar la aplicación

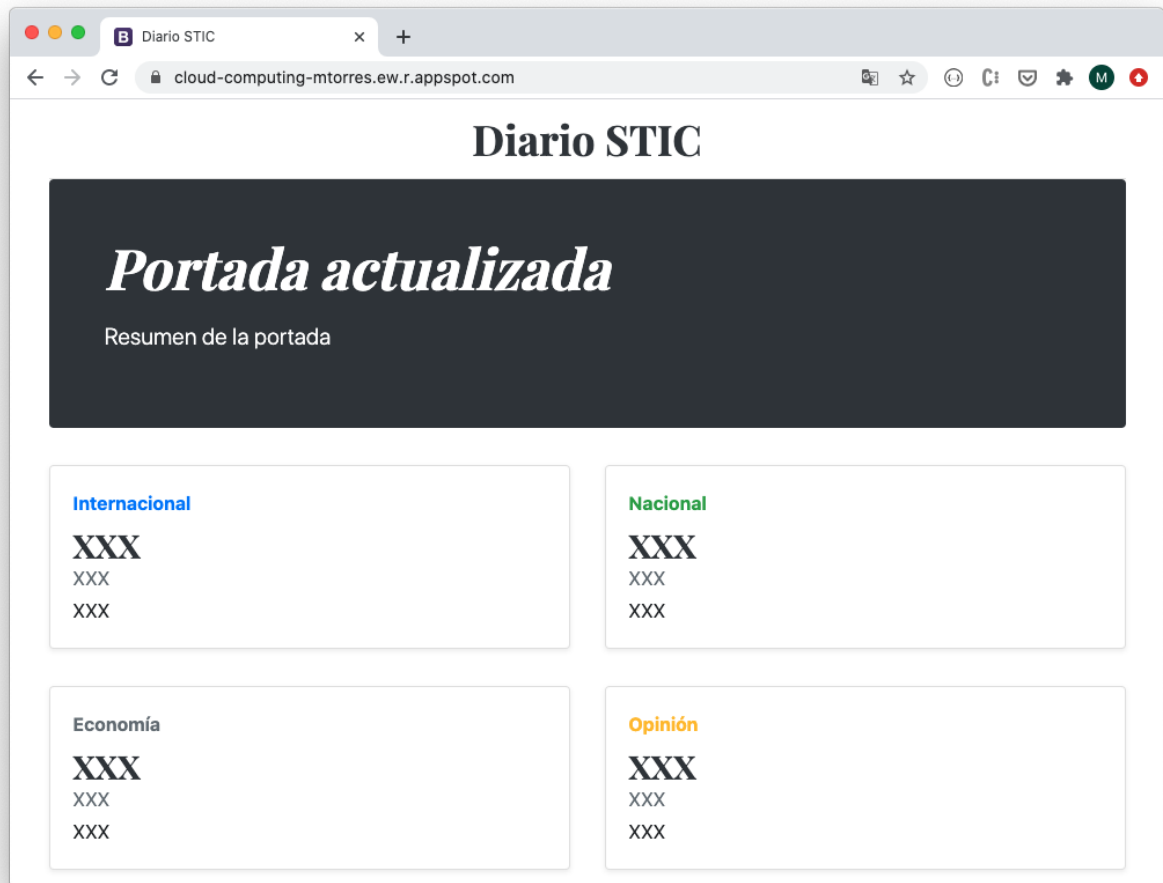
```
$ gcloud app deploy
```

Nos pedirá confirmación de despliegue sobre el descriptor (**app.yaml**) a usar, el origen (carpeta donde se colocó el repositorio) y el proyecto de destino. Tras confirmar, informará que se ha inicializado el Google Cloud SDK y comenzará el despliegue. La aplicación quedará almacenada en Google Cloud Storage.

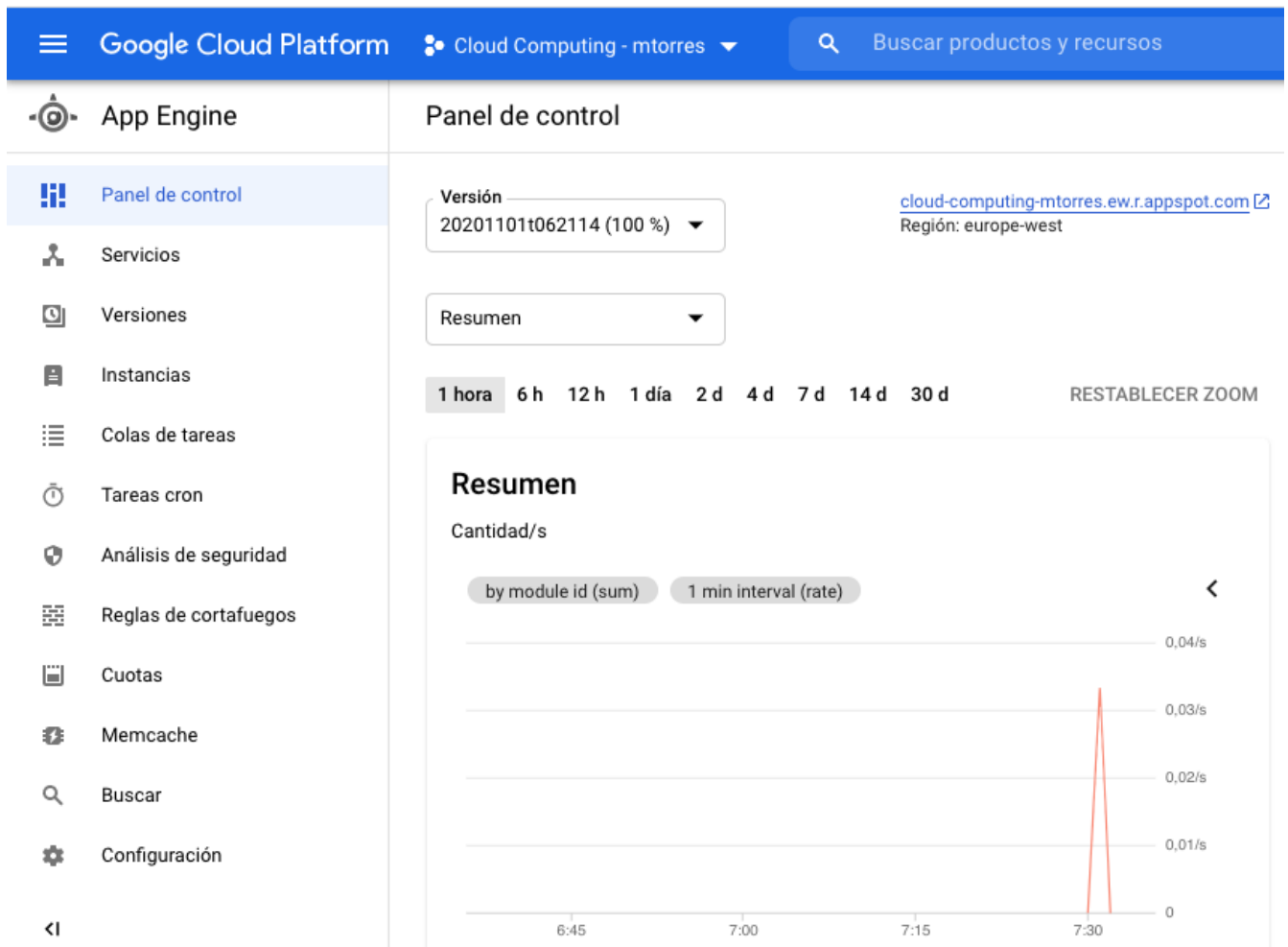
```
descriptor:    [/home/mtorres/diariostic/app.yaml]
source:        [/home/mtorres/diariostic]
target project: [cloud-computing-mtorres]
target service: [default]
target version: [20201101t062114]
target url:     [https://cloud-computing-mtorres.ew.r.appspot.com]
```

Do you want to continue (Y/n)?

Unos instantes más tarde podremos ver la aplicación ejecutando **gcloud app browse**. Esto nos proporcionará una URL para acceder a la aplicación si no hay un navegador predeterminado y se obtendrá un resultado como el de la figura.



La aplicación estará disponible en el **Panel de control** en el el bloque de **App Engine** del Menú de navegación. Mostrará las aplicaciones desplegadas en App Engine con un resumen del tráfico, resumen de las instancias desplegadas, estado de facturación, carga actual, errores y otra información de interés. No se perderán datos, se guardará su estado y posteriormente se podrá volver a habilitar.



Para evitar gastos innecesarios en el cupón de la cuenta de la asignatura, una vez hechas las pruebas inhabilitar la aplicación desde el menú **Configuración** de App Engine.

IMPORTANT

Google Cloud Platform Cloud Computing - mtorres

App Engine Configuración

Configuración de la aplicación Dominios personalizados Certificados SSL Remitentes de correo electrónico

Editar

Caducidad de la cookie de inicio de sesión de Google Predeterminada (1 día)

URLs de referencia API de Cuentas de Google

Inhabilitar aplicación

Al inhabilitar una aplicación, se detendrán todas las solicitudes activas, pero no se perderán datos ni estados. No obstante, se facturarán cargos si procede. Puedes volver a habilitar la aplicación cuando quieras.

Inhabilitar aplicación

Segmento de Cloud Storage predeterminado

Puedes usar hasta 5 GB de Cloud Storage con las aplicaciones de App Engine sin habilitar la facturación. [Más información](#)

[cloud-computing-mtorres.appspot.com](#)

Identity-Aware Proxy

Administra el acceso a los servicios alojados en App Engine.

INHABILITADA

[Configurar ahora](#)

4.1. Creación de la aplicación en App Engine en modo manual

En lugar de crear la aplicación desde el bloque App Engine del Menú de navegación es posible crear la aplicación directamente desde Cloud Shell con `gcloud init`.

Previamente, se habrá descargado el repositorio de código de la aplicación y se habrá incluido el descriptor de despliegue (`app.yaml`) en el directorio del código de la aplicación.

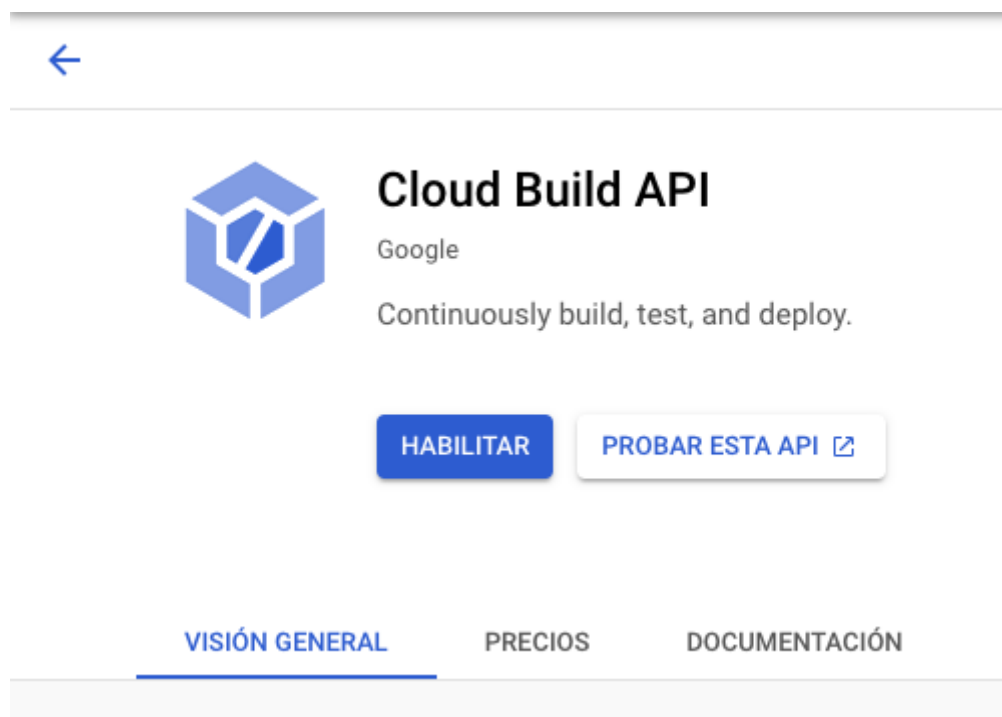
Para poder usar el SDK de Google Cloud en Cloud Shell, Google Cloud nos advierte que necesita incluir nuestras credenciales en Cloud Shell.

Como no se ha creado proyecto App Engine para la aplicación, nos pedirá si queremos crear proyecto nuevo o meter en existente. El valor introducido de tomará como ID del proyecto y no podrá ser cambiado posteriormente.

Después se desplegará con `gcloud app deploy`. Habrá que indicar una región de despliegue. Seleccionar `eu-west`.

Nos pedirá confirmación de despliegue sobre el descriptor (`app.yaml`) a usar, el origen (carpeta donde se colocó el repositorio) y el proyecto de destino. Tras confirmar, comenzará el despliegue y lo almacenará en Google Cloud Storage.

Para poder construir la aplicación es necesario que el proyecto tengan configurados permisos de acceso a la API de Cloud Build. Se nos pedirá que demos permiso a través de un enlace de activación de Cloud Build API en el proyecto. Se llegará a una pantalla como la siguiente.



Pulsar sobre **Habilitar**. A continuación, pedirá que se indique una cuenta donde vincular la facturación. Indicar la cuenta del cupón de la asignatura.

Una vez configurado el acceso a la API de Cloud Build, volver a Cloud Shell y volver a desplegar con `gcloud app deploy`.

4.2. Despliegue continuo

El despliegue automático en App Engine no es tan directo como en Vercel. Para más información, consultar la [documentación oficial](#).